Migration from CxxTest to doctest

This document describes how to migrate unit tests from CxxTest to doctest for aGrUM.

Why doctest?

Advantages over CxxTest

Feature	doctest	CxxTest
Dependencies	Header-only, no external tools	Requires Python for code generation
Compilation speed	Very fast (designed for speed)	Slower due to code generation
Error messages	Expression decomposition shows actual values	Basic assertion messages
Maintenance	Actively maintained	Essentially abandoned (last release 2016)
C++ standard	Full C++11/14/17/20 support	Limited modern C++ support
Build integration	Direct compilation, no preprocessing	Requires cxxtestgen step
Test organization	Subcases, BDD-style scenarios	Class-based only
Binary size	Minimal overhead	Larger test executables
Thread safety	Thread-safe by default	Manual synchronization needed

Key benefits for aGrUM

- **Simpler build process**: No more Python-based test generation step
- **Better CI integration**: Faster test compilation improves CI turnaround
- **Clearer failures**: When a test fails, doctest shows the actual vs expected values
- **Modern idioms**: Fixtures use standard C++ constructors/destructors

Using the migration script

The `migration.py` script automatically performs all necessary transformations.

Migrate a complete module

```
cd src/testunits
python3 migration.py --module module_CM
```

Migrate a single file

```
python3 migration.py module_CM/MyFileTestSuite.h
```

Migrate all files in a directory

```
python3 migration.py --all .
```

Transformations performed

The script performs the following transformations:

1. Class to struct conversion

```
// Before (CxxTest)
class GUM_TEST_SUITE(MyTest) {
    GUM_ACTIVE_TEST(TestName) { ... }
};

// After (doctest) - in module_XX/MyTestTestSuite.h
#undef GUM_CURRENT_SUITE
#undef GUM_CURRENT_MODULE
#define GUM_CURRENT_SUITE MyTest
#define GUM_CURRENT_MODULE XX

struct MyTestTestSuite {
    void testTestName() { ... }
};

GUM_TEST_ACTIF(TestName)
```

The migration script automatically extracts `GUM_CURRENT_SUITE` and `GUM_CURRENT_MODULE` from the file path: - `module_BN/BayesNetTestSuite.h` → `MODULE=BN`, `SUITE=BayesNet` - `module_BASE/DAGTestSuite.h` → `MODULE=BASE`, `SUITE=DAG`

The `GUM_TEST_ACTIF(TestName)` macro expands to:

```
TEST_CASE_FIXTURE(MyTestTestSuite, "[XX][MyTest] TestName") { testTestName(); }
```

To temporarily disable a test, use `GUM_TEST_INACTIF(TestName)` which generates a compilation warning:

```
#pragma message: /path/to/file.h:123: INACTIVATED TEST: TestName
```

2. Assertion macro replacement

CxxTest	doctest
`TS_ASSERT(x)`	`CHECK(x)`

CxxTest	doctest
`TS_ASSERT_EQUALS(a, b)`	`CHECK((a) == (b))`
`TS_ASSERT_DIFFERS(a, b)`	`CHECK((a) != (b))`
`TS_ASSERT_LESS_THAN(a, b)`	`CHECK((a) < (b))`
`TS_ASSERT_DELTA(a, b, d)`	`CHECK((a) == doctest::Approx(b).epsilon(d))`
`TS_ASSERT_THROWS(expr, Ex)`	`CHECK_THROWS_AS(expr, Ex)`
`TS_ASSERT_THROWS_NOTHING(expr)`	`CHECK_NOTHROW(expr)`
`TS_FAIL(msg)`	`FAIL(msg)`

3. setUp/tearDown conversion

The `setUp()` and `tearDown()` methods are converted to constructor and destructor:

```
// Before
void setUp() { ptr = new MyClass(); }
void tearDown() { delete ptr; }

// After
MyTestTestSuite() { ptr = new MyClass(); }
~MyTestTestSuite() { delete ptr; }
```

4. Complex expressions

Expressions with `||` or `&&` are automatically wrapped in extra parentheses:

```
// Before
CHECK(a == 1 || a == 2);

// After
CHECK((a == 1 || a == 2));
```

5. aGrUM-specific macro renaming

The `TS_GUM_*` macros are renamed to follow the `GUM_CHECK_*` convention:

CxxTest	doctest
`TS_GUM_ASSERT_THROWS_NOTHING(expr)`	`GUM_CHECK_ASSERT_THROWS_NOTHING(expr)`
`TS_GUM_TENSOR_ALMOST_EQUALS(p1, p2)`	`GUM_CHECK_TENSOR_ALMOST_EQUALS(p1, p2)`
`TS_GUM_TENSOR_ALMOST_EQUALS_DELTA(p1, p2, d)`	`GUM_CHECK_TENSOR_ALMOST_EQUALS_DELTA(p1, p2, d)`
`TS_GUM_TENSOR_ALMOST_EQUALS_SAMEVARS(p1, p2)`	`GUM_CHECK_TENSOR_ALMOST_EQUALS_SAMEVARS(p1, p2)`

CxxTest	doctest
`TS_GUM_TENSOR_QUASI_EQUALS(p1, p2)`	`GUM_CHECK_TENSOR_QUASI_EQUALS(p1, p2)`
`TS_GUM_TENSOR_DIFFERS(p1, p2)`	`GUM_CHECK_TENSOR_DIFFERS(p1, p2)`
`TS_GUM_TENSOR_DIFFERS_SAMEVARS(p1, p2)`	`GUM_CHECK_TENSOR_DIFFERS_SAMEVARS(p1, p2)`
`TS_GUM_TENSOR_SHOW_DELTA(p1, p2, d)`	`GUM_CHECK_TENSOR_SHOW_DELTA(p1, p2, d)`
`TS_GUM_SMALL_ERROR`	`GUM_SMALL_ERROR`
`TS_GUM_VERY_SMALL_ERROR`	`GUM_VERY_SMALL_ERROR`

aGrUM-specific macros

These aGrUM-specific macros are defined in `gumtest/AgrumTestSuite.h`:

Test registration

Define `GUM_CURRENT_SUITE` and `GUM_CURRENT_MODULE` once at the top of each test file, then use `GUM_TEST_ACTIF(TestName)` for active tests:

```
// At the top of module_BN/BayesNetTestSuite.h:
#undef GUM_CURRENT_SUITE
#undef GUM_CURRENT_MODULE
#define GUM_CURRENT_SUITE BayesNet
#define GUM_CURRENT_MODULE BN

// Then for each test:
GUM_TEST_ACTIF(Constructor)
GUM_TEST_ACTIF(CopyConstructor)

// To temporarily disable a test (generates a compilation warning):
GUM_TEST_INACTIF(FlakyTest)

// Instead of:
TEST_CASE_FIXTURE(BayesNetTestSuite, "[BN][BayesNet] Constructor") { testConstructor(); }
```

Assertions

- `GUM_CHECK_ASSERT_THROWS_NOTHING` - Displays gum::Exception error details on failure
- `GUM_CHECK_TENSOR_ALMOST_EQUALS` - Tensor comparison with tolerance
- `GUM_CHECK_TENSOR_DIFFERS` - Tensor difference check
- `GUM_SMALL_ERROR` / `GUM_VERY_SMALL_ERROR` - Tolerance constants (1e-5 / 1e-10)

After migration

1. Verify that the file compiles:

   ```
   act test release aGrUM -t <module> -m all
   ```

2. Verify that tests pass:

   ```
   ./build/aGrUM/release/gumTest --test-case="*[MODULE]*"
   ```

Module timing summary

A custom doctest reporter automatically displays a timing summary per module at the end of each test run:

Module Timing Summary		
Module	Tests	Time (s)
BN	692	33.16
BASE	663	4.43
TOTAL	1355	37.60

This reporter is implemented in `gumtest/ModuleTimingReporter.h` and registered as a doctest listener.

doctest command-line options

```
# List all tests
./gumTest --list-test-cases

# Run tests for a specific module
./gumTest --test-case="*[BN]*"

# Run a specific test
./gumTest --test-case="*Constructor*"

# Run with verbose output
./gumTest -s

# Run with per-test timing
./gumTest -d
```

CLion integration

CLion has native support for doctest. Here's how to configure it:

1. Open the CMake project

Open the `aGrUM-dev` folder as a CMake project. CLion will automatically detect the doctest framework.

2. Create a test run configuration

Run → Edit Configurations → + → CMake Application

Configure as follows: - **Name**: `gumTest` - **Target**: `gumTest` - **Program arguments**: (optional, e.g., `--test-case="*[BN]*"`) - **Working directory**: `$ProjectFileDir$/src/testunits`

3. Run tests

Several options are available: - Click the green "Run" icon next to any `TEST_CASE_FIXTURE` in the editor - Use the run configuration created above - Press **Ctrl+Shift+F10** (or **Ctrl+Shift+R** on macOS) on a test to run it directly

4. Filter tests

Use program arguments to filter which tests to run:

```
--test-case="*[BN]*"          # Run tests from the BN module
--test-case="*BayesNet*"      # Run tests containing "BayesNet"
--list-test-cases             # List all available tests
```

5. Test results window

CLion displays results in the **Run** tool window with: - Tree view of all tests - Execution time per test - Click-to-navigate to source code on failure - Re-run failed tests only

Optional: CMake Presets

To synchronize CLion with `act` build configurations, create a `CMakePresets.json` in the project root:

```
{
  "version": 3,
  "configurePresets": [
    {
      "name": "release",
      "binaryDir": "${sourceDir}/build/aGrUM/release",
      "cacheVariables": {
        "CMAKE_BUILD_TYPE": "Release",
        "BUILD_ALL": "ON"
      }
```

```
    },
    {
      "name": "debug",
      "binaryDir": "${sourceDir}/build/aGrUM/debug",
      "cacheVariables": {
        "CMAKE_BUILD_TYPE": "Debug",
        "BUILD_ALL": "ON"
      }
    }
  ]
}
```

Then in CLion: **File → Settings → Build, Execution, Deployment → CMake → Add preset**.